

3. Metodyka

W rozdziale przedstawiono metody zastosowane przy wytworzeniu platformy MoodFlow w oparciu o wymagania inżynierii oprogramowania oraz wybrane technologie informatyczne. Opisano metodyki projektowe, organizację pracy zespołu, wzorce architektoniczne, dobór technologii oraz narzędzia wspierające proces wytwarzania.

3.1. Metodyka pracy zespołowej

Wytwarzanie platformy oparto o iteracyjne podejście wywodzące się z filozofii Agile, ze szczególnym uwzględnieniem elementów metodyki **Scrum** oraz tablicy zadań w stylu **Kanban**. Wybór metodyki zwinnej był podyktowany kilkoma czynnikami. Po pierwsze, na początku projektu nie wszystkie wymagania były znane w sposób precyzyjny i pojawiała się konieczność ich uściślenia w trakcie realizacji. Po drugie, zespół składał się z osób o różnych specjalizacjach, co wymagało regularnej komunikacji i transparentności postępów. Po trzecie, krótkie iteracje pozwalały na bieżącą walidację rozwiązań w kontekście wymagań pracy inżynierskiej oraz konsultacji z promotorem.

Praca przebiegała w dwutygodniowych iteracjach. Na początku każdej iteracji ustalano zakres zadań do wykonania na podstawie listy *backlog*, w trakcie iteracji prowadzono krótkie spotkania synchronizacyjne, a na zakończenie weryfikowano efekty. Tablica zadań w narzędziu do zarządzania projektami obrazowała aktualny stan każdego zadania w pięciu kolumnach: *Backlog*, *To Do*, *In Progress*, *Review* oraz *Done*.

Alternatywą rozważaną była metodyka **Waterfall**, jednak została odrzucona ze względu na ograniczenia wynikające z liniowej kolejności faz oraz brak elastyczności w przypadku zmiany wymagań w trakcie projektu. Charakter pracy inżynierskiej, w której wymagania funkcjonalne dojrzewały równolegle z postępami implementacji, wymagał podejścia adaptacyjnego.

3.2. Metodyka inżynierii wymagań

Specyfikację wymagań przeprowadzono zgodnie z dobrymi praktykami opisanymi w przewodniku Babok Guide 3.0, według którego wymaganie funkcjonalne stanowi „użyteczną reprezentację potrzeby oraz skupia się na wartości, jaką można dostarczyć, gdy wymaganie zostanie spełnione”. Wymagania zostały podzielone na cztery poziomy: biznesowe (cel projektu), interesariuszy (oczekiwania ról), systemowe (funkcjonalne i niefunkcjonalne) oraz przejściowe (związane z wdrożeniem).

Każde wymaganie spełniało zasadę **SMART** — było konkretne (Specific), mierzalne (Measurable), osiągalne (Achievable), istotne (Relevant) oraz określone w czasie (Time-bound). Wymagania funkcjonalne formułowano w postaci *user stories* według wzorca: „Jako [rola], chciałbym [czynność], aby [cel]”, a następnie rozszerzano o opis, kryteria akceptacji oraz wykonawcę.

Do priorytetyzacji wymagań zastosowano metodę **MoSCoW**, opisaną szczegółowo w rozdziale 2.1. Pozwoliła ona na jednoznaczne rozdzielenie funkcjonalności krytycznych (Must Have) od pomocniczych (Should Have, Could Have) oraz świadomie wykluczonych (Won't Have).

Najbardziej krytyczne przepływy interakcji opisano w postaci *Use Case* w formie tabelarycznej, zawierającej pola: identyfikator i nazwę przypadku użycia, opis, aktorów, warunki początkowe (preconditions), wyzwalacz (trigger), główny przepływ zdarzeń (main flow), przepływy alternatywne (alternative flow), wyjątki (exceptions), warunki końcowe (postconditions) oraz powiązane wymagania.

3.3. Modelowanie systemu

Do modelowania systemu zastosowano notację **UML** (Unified Modeling Language) zgodnie z wymaganiami inżynierii oprogramowania. Diagramy przygotowano przy użyciu

narzędzia Mermaid — języka opisu diagramów w formie tekstowej, dzięki czemu źródło każdego diagramu jest wersjonowane razem z kodem aplikacji w systemie kontroli wersji.

W projekcie sporządzono następujące diagramy:

- **Diagram przypadków użycia** — przegląd interakcji aktorów (pracownik, HR, administrator firmy, właściciel platformy, system mailowy) z funkcjami systemu;
- **Diagram klas** — struktura modułów backendu z ich zależnościami i relacjami wstrzykiwania;
- **Diagram ERD** — relacyjny model bazy danych z dziesięcioma encjami głównymi (organizations, users, departments, assessments, questions, answer_options, assessment_assignments, assessment_results, user_responses, audit_logs, notifications);
- **Diagram sekwencji** — przepływ danych przy wypełnianiu testu psychologicznego, od pobrania definicji testu po zapis wyniku;
- **Diagram maszyny stanów** — cykl życia konta użytkownika (PENDING, ACTIVE, REJECTED, SUSPENDED) oraz organizacji (PENDING, ACTIVE, BLOCKED, REJECTED);
- **Diagram komponentów** — relacje pomiędzy komponentami systemu (backend, dwa frontendy, baza danych, system mailowy) z zaznaczeniem protokołów komunikacji (HTTPS, REST, TCP, SMTP);
- **Diagram wdrożenia** — rozmieszczenie komponentów na platformie chmurowej z zaznaczeniem zależności pomiędzy węzłami;
- **Diagram BPMN** — pełen proces biznesowy od rejestracji firmy, przez zatwierdzenie, dodanie pracownika, aż po wypełnienie pierwszego testu i analizę wyniku w panelu HR.

3.4. Architektura systemu

Platformę zaprojektowano jako **monolit modułarny** w warstwowej architekturze typu *layered architecture*. Wybór tego wzorca uzasadniono trzema czynnikami: prostotą wdrożenia w porównaniu do mikrousług, dopasowaniem do skali projektu inżynierskiego oraz czytelnością kodu dla osób oceniających pracę.

Backend zorganizowano w niezależne moduły domenowe (*auth, users, organizations, departments, assessments, responses, results, analytics, admin, notifications, audit, health*), z których każdy zawiera własny kontroler (warstwa prezentacji), serwis (warstwa logiki biznesowej), repozytoria (warstwa dostępu do danych) oraz definicje encji. Podejście to wynika z metodyki Domain-Driven Design w jej uproszczonej formie i pozwala na czytelne mapowanie kodu na obszary biznesowe.

Architektura systemu jest **wielodostępna (multi-tenant)** w modelu *row-level multi-tenancy* — wszystkie dane wszystkich firm znajdują się w jednej bazie danych, a izolacja realizowana jest przez filtrowanie zapytań po identyfikatorze organizacji pobieranym z tokenu uwierzytelniającego. Alternatywne modele (osobna baza per firma, osobny schemat) odrzucono jako zbyt kosztowne operacyjnie dla skali projektu.

Komunikacja między frontendami a backendem odbywa się przez **REST API** z formatem JSON, przy użyciu metod HTTP zgodnie z konwencją: GET (odczyt), POST (utworzenie), PATCH (modyfikacja), DELETE (usunięcie). Endpointy nazywane są zgodnie z konwencją zasobów w liczbie mnogiej, a zwracane odpowiedzi mają ujednoliconą strukturę.

3.5. Wybrane technologie informatyczne

Każda technologia została dobrana w oparciu o wymagania projektu, dojrzałość ekosystemu oraz wartość edukacyjną dla pracy inżynierskiej.

3.5.1. Język programowania — TypeScript

Cały kod aplikacji został napisany w języku **TypeScript** — nadbudowie nad JavaScriptem dodającej system typów statycznych weryfikowanych podczas kompilacji. Wybór tego języka zapewnia wykrywanie znacznej części błędów na etapie kompilacji, spójność typów end-to-end pomiędzy backendem a frontendami oraz zgodność z aktualnym standardem branżowym w nowoczesnym *web developmencie*.

3.5.2. Backend — Node.js i NestJS

Warstwa serwerowa zbudowana została w środowisku Node.js w wersji 20 z użyciem frameworka **NestJS 11**. NestJS wprowadza do ekosystemu JavaScript koncepcje znane z dojrzałych frameworków, takich jak modułowość, wstrzykiwanie zależności (*dependency injection*), dekoratory oraz wyraźny podział na warstwy. Ekosystem oficjalnych modułów (@nestjs/jwt, @nestjs/passport, @nestjs/typeorm, @nestjs/throttler) pozwolił na szybkie wdrożenie kluczowych funkcji bez korzystania z niesprawdzonych bibliotek zewnętrznych.

3.5.3. Trwałość danych — PostgreSQL z TypeORM

Jako system zarządzania bazą danych zastosowano **PostgreSQL 16**. Komunikacja z bazą realizowana jest poprzez **TypeORM** — bibliotekę typu Object-Relational Mapping zintegrowaną z NestJS. PostgreSQL wybrano ze względu na natywne wsparcie typu JSONB (wykorzystywanego dla tabel *audit_logs* i *assessment_results*), pełniejszą zgodność ze standardem SQL niż MySQL oraz dojrzałość i stabilność rozwijanego od ponad trzydziestu lat projektu.

3.5.4. Frontend pracownika — React i Vite

Aplikacja przeznaczona dla pracownika została zrealizowana w bibliotece **React 18** z wykorzystaniem narzędzia budującego **Vite 8**. Stan aplikacji zarządzany jest przez **Redux Toolkit**, a routing realizuje biblioteka **React Router 6**. Vite zapewnia czas startu serwera deweloperskiego poniżej sekundy oraz wsparcie dla budowania aplikacji jako Progressive Web App poprzez wtyczkę vite-plugin-pwa.

3.5.5. Frontend administracyjny — Next.js

Panele administracyjne (administrator firmy, HR, właściciel platformy) zostały zbudowane przy użyciu frameworka **Next.js 16** — nadbudowy nad Reactem rozwijanej przez firmę Vercel. Next.js wprowadza renderowanie po stronie serwera, automatyczne dzielenie kodu, routing oparty o strukturę folderów oraz model *App Router*.

3.5.6. Stylowanie interfejsu — Tailwind CSS

Wszystkie warstwy prezentacji aplikacji wykorzystują **Tailwind CSS** w wersji 4 — framework typu *utility-first*, w którym style są aplikowane jako klasy bezpośrednio w kodzie. Predefiniowane skale kolorów, marginesów i typografii eliminują niespójności wynikające z ręcznego pisania CSS. System tokenów CSS pozwolił na zaimplementowanie trybu jasnego i ciemnego z minimalną ingerencją w komponenty.

3.5.7. Wizualizacja danych — Recharts oraz SVG

W panelu HR wykresy (rozkład nasilenia objawów, trendy w czasie, statystyki działowe) zaimplementowano przy użyciu biblioteki **Recharts**. W aplikacji pracownika wykresy indeksu dobrostanu i historii wyników wykonano bezpośrednio w technologii **SVG**, co umożliwia precyzyjną kontrolę nad wyglądem (krzywe Bezierra dla wygładzonej linii, gradienty pod krzywą, animowane wartości).

3.5.8. Progressive Web App

Aplikacja pracownika została zaimplementowana jako Progressive Web App z wykorzystaniem wtyczki vite-plugin-pwa opartej na bibliotece Workbox. Implementacja zawiera manifest aplikacji, ikony w formacie PNG (192×192, 512×512, 180×180 dla

apple-touch-icon), Service Worker buforujący zasoby statyczne z polityką cache-first oraz mechanizm informowania użytkownika o dostępności nowej wersji.

3.5.9. Bezpieczeństwo aplikacji

W warstwie bezpieczeństwa wykorzystano zestaw sprawdzonych bibliotek branżowych: **bcrypt** z dwunastoma rundami soli do hashowania haseł, **JSON Web Tokens** z różnymi sekretami dla tokenów dostępu i odświeżania, **Helmet** dla nagłówków bezpieczeństwa HTTP, **cookie-parser** dla obsługi ciasteczek HttpOnly, **class-validator** dla walidacji danych wejściowych oraz **@nestjs/throttler** dla ograniczenia liczby prób logowania (10 prób na 15 minut) i rejestracji (5 prób na 15 minut).

3.5.10. Konteneryzacja i wdrożenie — Docker i Railway

Wszystkie komponenty systemu działają w izolowanych kontenerach **Docker**, a orkiestrację realizuje narzędzie **Docker Compose** w dwóch konfiguracjach — rozwojowej i produkcyjnej. Aplikacja produkcyjna została wdrożona na platformie chmurowej **Railway**, która zapewnia automatyczne udostępnianie certyfikatów TLS Let's Encrypt, managed PostgreSQL z dziennymi snapshotami oraz natywne wsparcie dla aplikacji w kontenerach Docker.

3.6. Narzędzia wspierające proces wytwarzania

Proces wytwarzania platformy wspierany był następującymi narzędziami:

Tabela 3.1 Narzędzia wspierające proces wytwarzania (źródło: opracowanie własne)

Kategoria	Narzędzie	Zastosowanie
Kontrola wersji	Git, GitHub	Wersjonowanie kodu, współpraca w zespole, przegląd zmian
Środowisko rozwojowe	Visual Studio Code	Edytor kodu z dodatkami ESLint, Prettier, Tailwind IntelliSense
Zarządzanie projektem	GitHub Projects	Tablica zadań, lista <i>backlog</i> , śledzenie postępów
Komunikacja zespołowa	Discord, Google Meet	Spotkania zespołowe, czat roboczy, rozmowy z promotorem
Diagramy	Mermaid	Tworzenie diagramów UML i BPMN w formacie tekstowym
Projektowanie UI	Pencil, Figma	Wstępne makiety interfejsu, system wizualny aplikacji
Testy ręczne API	Postman, Thunder Client	Weryfikacja endpointów REST w trakcie rozwoju
Środowisko pisania pracy	Microsoft Word, Google Docs	Redagowanie dokumentu pracy inżynierskiej
Konteneryzacja	Docker, Docker Compose	Izolowane środowisko uruchomieniowe
Hosting	Railway	Wdrożenie produkcyjne, hosting bazy danych

3.7. Procedury testowania

W ramach pracy zaplanowano dwa rodzaje testów. **Testy klikalne** (manualne) obejmowały weryfikację poprawności wprowadzania danych w formularzach (rejestracja, logowanie, zmiana hasła), kompletności wypełnienia testów psychologicznych, działania

mechanizmu zatwierdzania kont, przypisywania testów oraz generowania raportów w panelu HR z uwzględnieniem progu anonimizacji $k=5$. Każdy z głównych przepływów aplikacji został przetestowany na trzech kontach o różnych rolach: pracownik, administrator firmy oraz właściciel platformy.

Testy nieklikalne obejmowały weryfikację interfejsu użytkownika pod kątem responsywności na trzech klasach urządzeń (telefon, tablet, komputer), zgodności trybu jasnego i ciemnego, czytelności komunikatów błędów oraz zachowania przy braku połączenia z serwerem. Sprawdzano również odpowiedź backendu na dane wejściowe niespełniające reguł walidacji oraz reakcję na próbę dostępu do zasobów bez wymaganych uprawnień.

Procedury testowania prowadzono iteracyjnie w trakcie każdej dwutygodniowej iteracji, a wyniki testów stanowiły kryterium zamknięcia zadania.

3.8. Archiwizacja i bezpieczeństwo danych

Archiwizacja danych realizowana jest przez managed PostgreSQL platformy Railway z mechanizmem automatycznych snapshotów wykonywanych raz na dobę. Snapshoty są przechowywane przez 30 dni, co umożliwia odtworzenie stanu bazy z dowolnego punktu w tym okresie. Wszystkie dane operacyjne (konta użytkowników, organizacje, wyniki testów, dziennik audytu, powiadomienia) przechowywane są w jednej bazie danych, w postaci znormalizowanej zgodnie z zasadami trzeciej postaci normalnej (3NF).

Bezpieczeństwo danych realizowane jest na kilku poziomach. Komunikacja sieciowa odbywa się wyłącznie przez szyfrowane połączenie HTTPS z certyfikatem TLS. Hasła użytkowników przechowywane są wyłącznie w postaci hashy bcrypt z dwunastoma rundami soli. Tokeny uwierzytelniające przekazywane są w ciasteczkach typu HttpOnly z atrybutami Secure i SameSite=Strict, co zabezpiecza przed atakami klasy XSS i CSRF. Dostęp do bazy danych jest możliwy wyłącznie z wewnętrznej sieci platformy chmurowej, bez publicznego adresu sieciowego. Dane wrażliwe dotyczące zdrowia psychicznego są dodatkowo anonimizowane na poziomie raportów według zasady k -anonymity z progiem $k=5$.

Każda akcja administracyjna (zatwierdzenie konta, blokada, zmiana roli, próba logowania) rejestrowana jest w dzienniku audytu (*audit log*), co realizuje wymóg rozliczalności wynikający z RODO oraz umożliwia rekonstrukcję incydentów bezpieczeństwa.